

Future directions of Dynamic Gol

Koko Muroya
(NII)

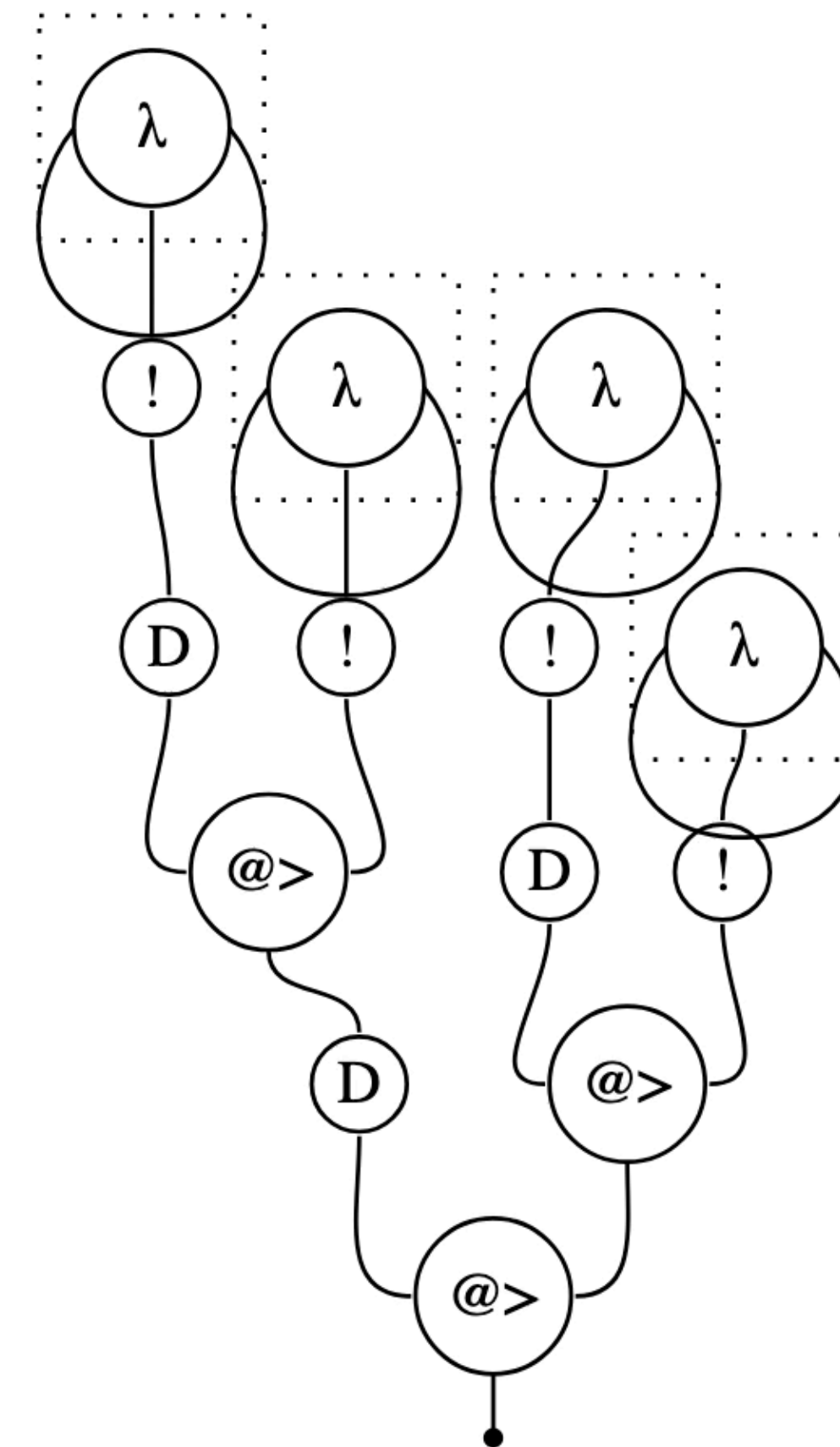
Dynamic GoI [M.+, LMCS'19]

- operational semantics for functional programs
(esp. lambda-calculus with various evaluation strategies)
- token passing + graph rewriting
 - programs as graphs
 - redex search as token passing
 - reduction as graph rewriting

Background

Dynamic Gol [M.+, LMCS'19]

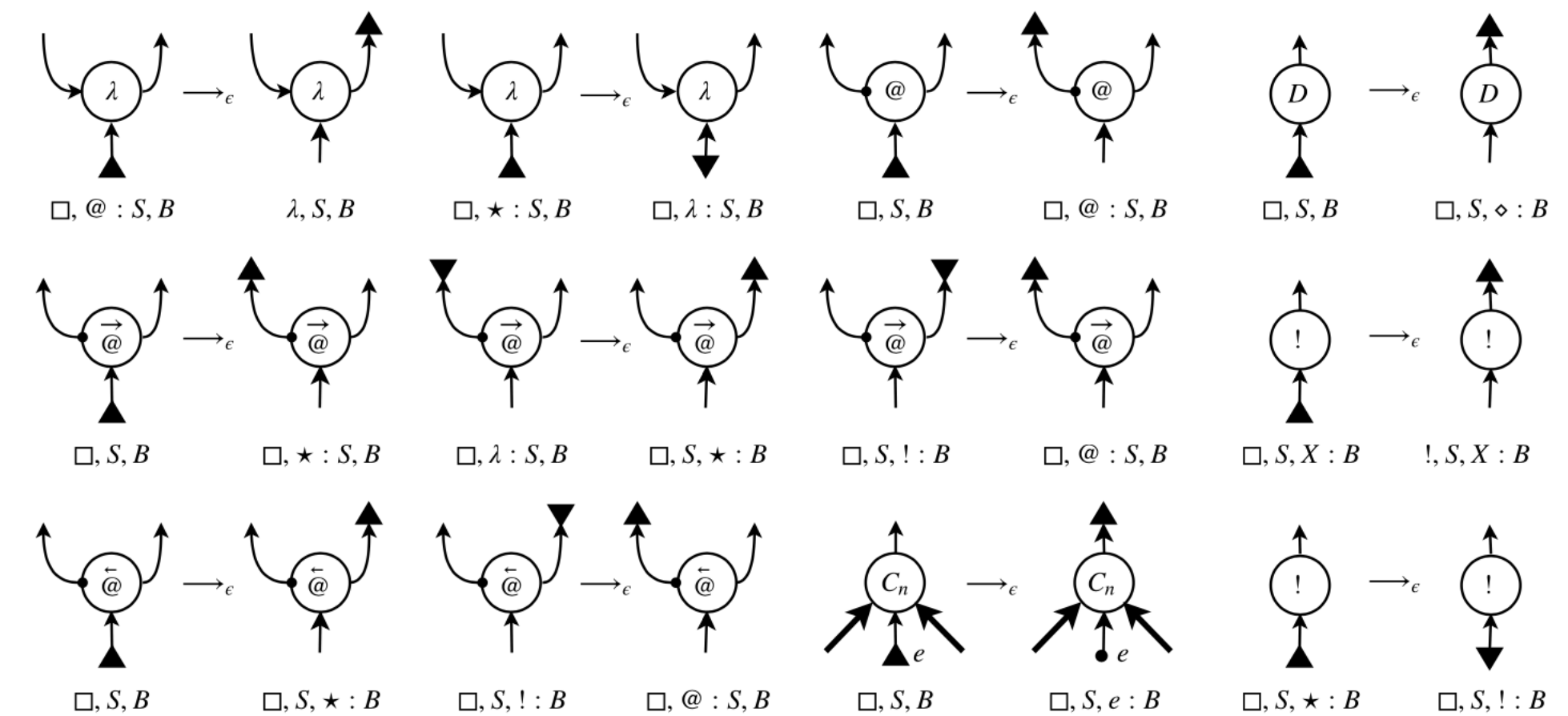
- $((\lambda x . x) (\lambda y . y)) ((\lambda z . z) (\lambda w . w))$

$$(\lambda y . y) ((\lambda z . z) (\lambda w . w))$$
$$(\lambda y . y) \ (\lambda w . w)$$
 $\lambda w . w$ 

- <https://koko-m.github.io/Gol-Visualiser/>

Dynamic GoI [M.+, LMCS'19]

- operational semantics for functional programs
(esp. lambda-calculus with various evaluation strategies)
- token passing + graph rewriting
 - programs as graphs
 - redex search as token passing
 - reduction as graph rewriting

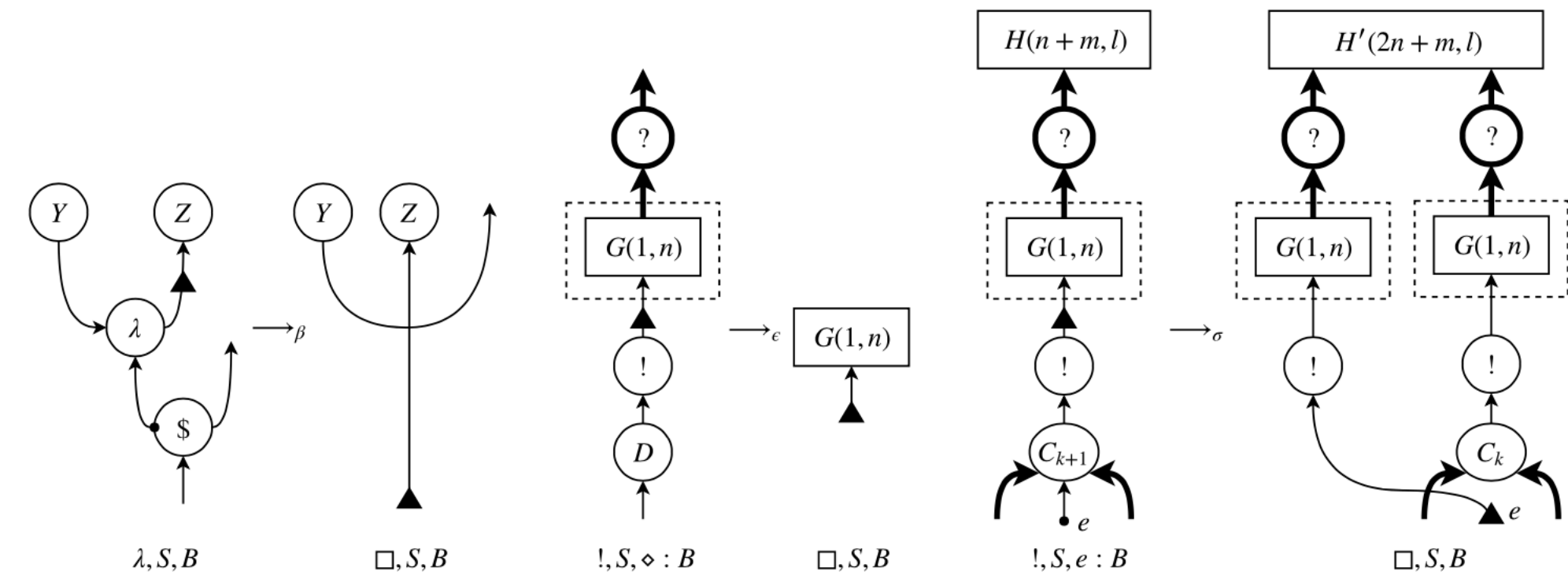


where $X \neq \star$.

Figure 5: Pass Transitions

Dynamic GoI [M.+, LMCS'19]

- operational semantics for functional programs
(esp. lambda-calculus with various evaluation strategies)
- token passing + graph rewriting
 - programs as graphs
 - redex search as token passing
 - reduction as graph rewriting

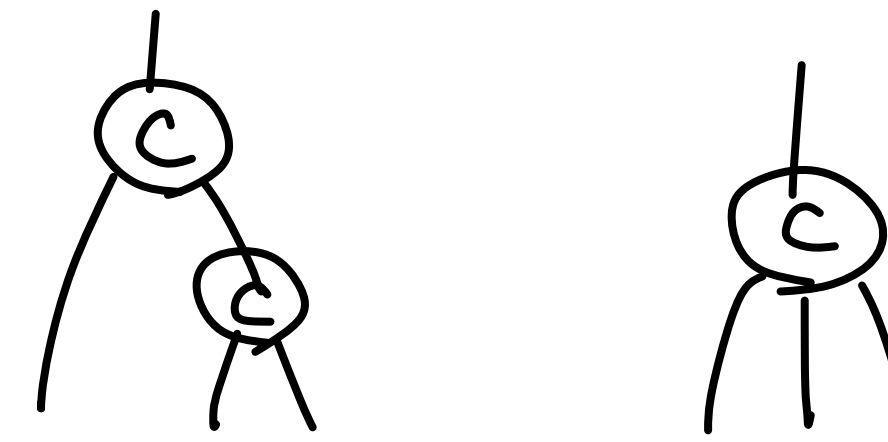


where $Y \in \mathcal{L}$, $Z \in \mathcal{L}$, $\$ \in \{\textcircled{!}, \overrightarrow{\textcircled{!}}, \overleftarrow{\textcircled{!}}\}$, and $G(1, n)$ is any graph.

Figure 6: Rewrite Transitions

Dynamic GoI [M.+ , LMCS'19]

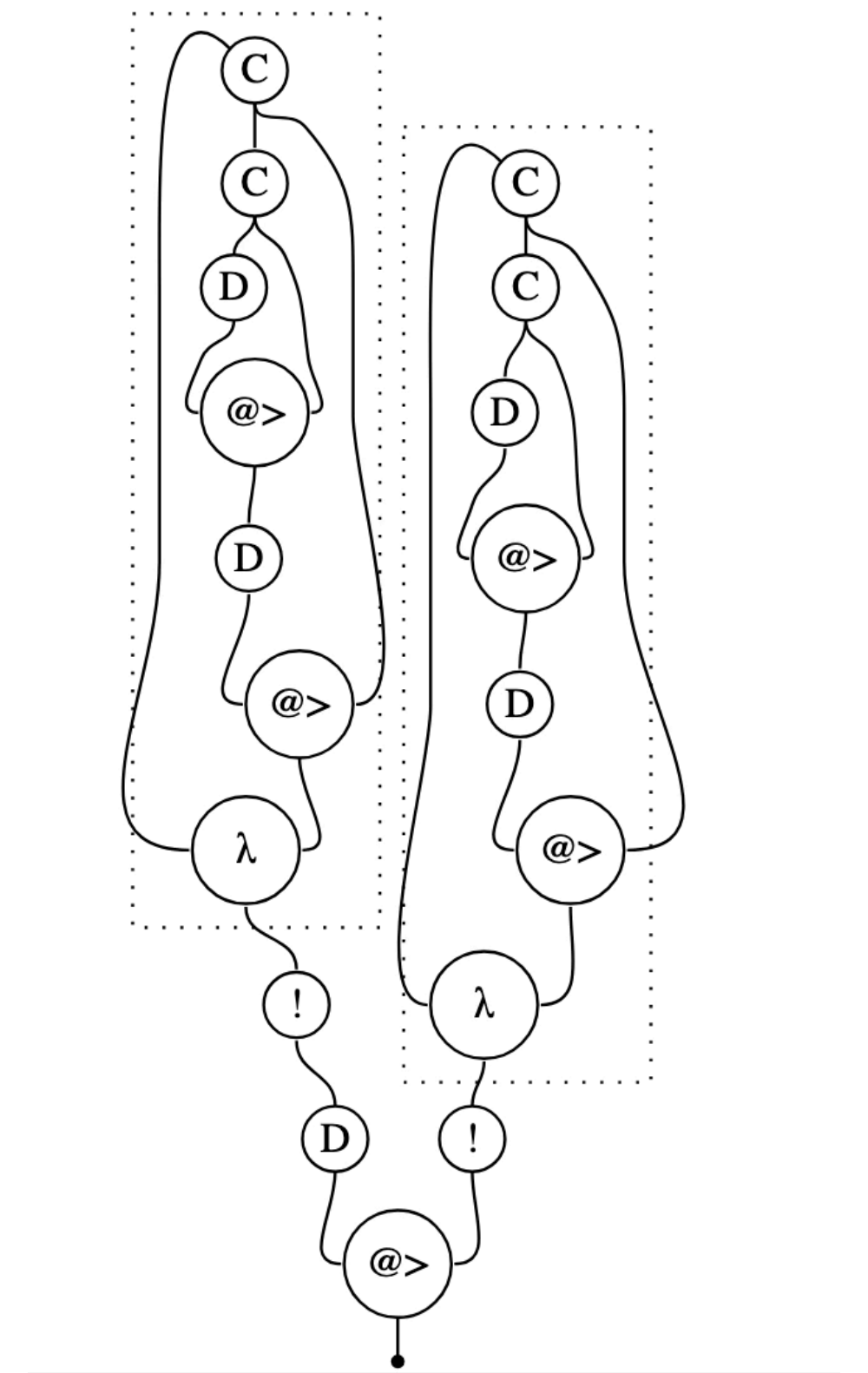
- token passing + graph rewriting
 - “rewrite whenever possible”
 -  accurate (“efficient” a la [Accattoli+, '14]) time efficiency
 - $\mathcal{O}(|t| \cdot |\rightarrow_{\beta}|)$ time for evaluating t
 - caveat: need an efficient representation of variable sharing
 - e.g. $\lambda f. \lambda x. f (f x)$



Background

Dynamic GoI [M.+ , LMCS'19]

- token passing + graph rewriting
 - “rewrite whenever possible”
 -  accurate (“efficient” a la [Accattoli+, '14]) time efficiency
 - $\mathcal{O}(|t| \cdot |\rightarrow_{\beta}|)$ time for evaluating t
 - space efficiency can be bad
 - e.g. $(\lambda x . x x x) (\lambda x . x x x)$



Dynamic GoI [M.+ , LMCS'19] with flexible interleaving

- token passing + graph rewriting
 - “rewrite whenever possible”
 -  accurate (“efficient” a la [Accattoli+, '14]) time efficiency
 - space efficiency can be bad
- Question: how about other interleaving strategies?
 - allowing choice between token passing and graph rewriting
 - nondeterministic semantics, various time/space efficiency

Dynamic GoI [M.+ , LMCS'19] with flexible interleaving

- token passing + graph rewriting
 - “rewrite whenever possible”
 -  accurate (“efficient” a la [Accattoli+, '14]) time efficiency
 - space efficiency can be bad
- Question: how about other interleaving strategies?
 - “never rewrite; token passing only”
 - so-called *GoI semantics* [Girard '89] [Danos & Regnier '99] [Mackie '95]
 - <https://koko-m.github.io/GoI-Visualiser/>

Dynamic GoI [M.+ , LMCS'19] with flexible interleaving

- token passing + graph rewriting
 - “rewrite whenever possible”
 -  accurate (“efficient” a la [Accattoli+, '14]) time efficiency
 - space efficiency can be bad
- Question: how about other interleaving strategies?
 - “never rewrite; token passing only”
 - good space efficiency (folklore)
 - time efficiency can be bad, due to repeated evaluation

Dynamic GoI [M.+ , LMCS'19] with flexible interleaving

- token passing + graph rewriting
 - “rewrite whenever possible”
 -  accurate (“efficient” a la [Accattoli+, '14]) time efficiency
 - space efficiency can be bad
- Question: how about *other* interleaving strategies?

Dynamic GoI [M.+ , LMCS'19] with flexible interleaving

- token passing + graph rewriting
 - “rewrite whenever possible”
 -  accurate (“efficient” a la [Accattoli+, '14]) time efficiency
 - space efficiency can be bad
- Question: how about *other* interleaving strategies?
 - “rewrite for the first k times”
 - “only rewrite when the graph does not expand”
 - (initial discussion with G. Vanoni back in 2019...)

Dynamic GoI [M.+ , LMCS'19] with flexible interleaving

- token passing + graph rewriting
 - “rewrite whenever possible”
 -  accurate (“efficient” a la [Accattoli+, '14]) time efficiency
 - space efficiency can be bad
- Question: how about *other* interleaving strategies?
 - “rewrite for the first k times”
 - “only rewrite when the graph does not expand”
 - → could express the time-space trade-off, or adjust efficiency

Detour: my recent work on term rewriting

Term evaluation and refinement systems [M.+, FLOPS'24]

- systems with two kinds of rewriting
 - *evaluation* ($l \rightarrow r$) as specification of operational semantics
 - *refinement* ($l \Rightarrow r$) as optimisation
- Goal: judge if *refinement* is “correct” wrt. *evaluation*
 - namely, prove *contextual improvement* [Sands '96]

$$\forall C . \forall (l \Rightarrow r) . C[l\theta] \rightarrow^k v \implies C[r\theta] \rightarrow^m v \wedge k \geq m$$

Detour: my recent work on term rewriting

Term evaluation and refinement systems [M.+, FLOPS'24]

- e.g. call-by-value lambda-calculus as a core of functional programming

- beta-reduction as *computation step*

$$(\lambda x . M) V \rightarrow M[V/x]$$

- eta-reduction as *optimisation*

$$\lambda x . V x \Rightarrow V$$

- Goal: judge if *refinement* is “correct” wrt. *evaluation*

- namely, prove *contextual improvement* [Sands '96]

$$\forall C . \forall (l \Rightarrow r) . C[l\theta] \rightarrow^k v \implies C[r\theta] \rightarrow^m v \wedge k \geq m$$

Detour: my recent work on term rewriting

Term evaluation and refinement systems [M.+, FLOPS'24]

- e.g. call-by-value lambda-calculus as a core of functional programming

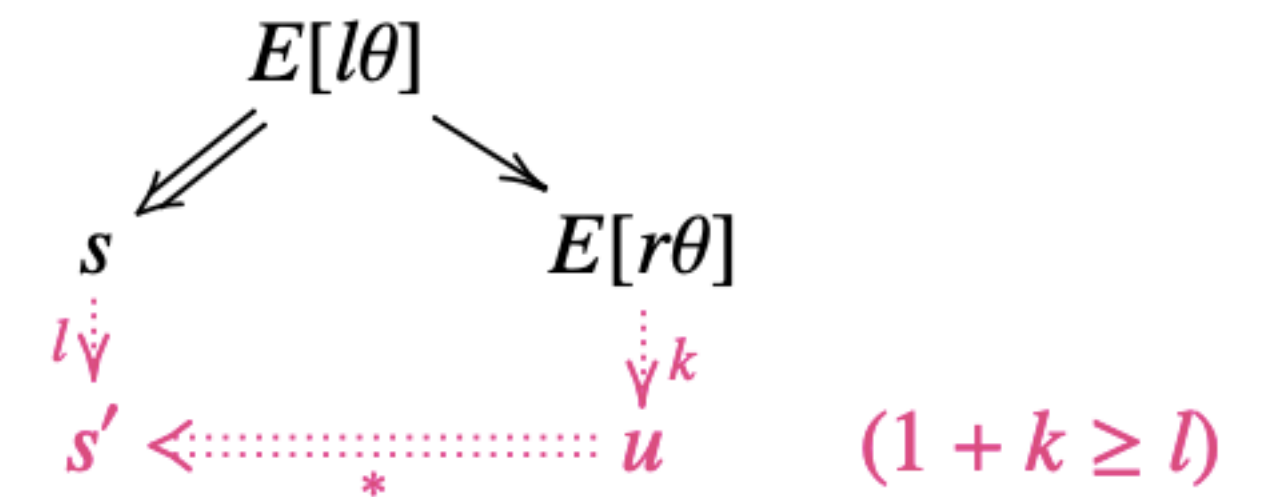
- beta-reduction as *computation step*

$$(\lambda x . M) V \rightarrow M[V/x]$$

- eta-reduction as *optimisation*

$$\lambda x . V x \Rightarrow V$$

- Goal: judge if *refinement* is “correct” wrt. *evaluation*



- Key sufficient condition: *local coherence* [Huet '80] [Aoto & Toyama '12]

can be checked by *critical pair analysis*

Detour: my recent work on term rewriting

Term evaluation and refinement systems [M., FLOPS'24]

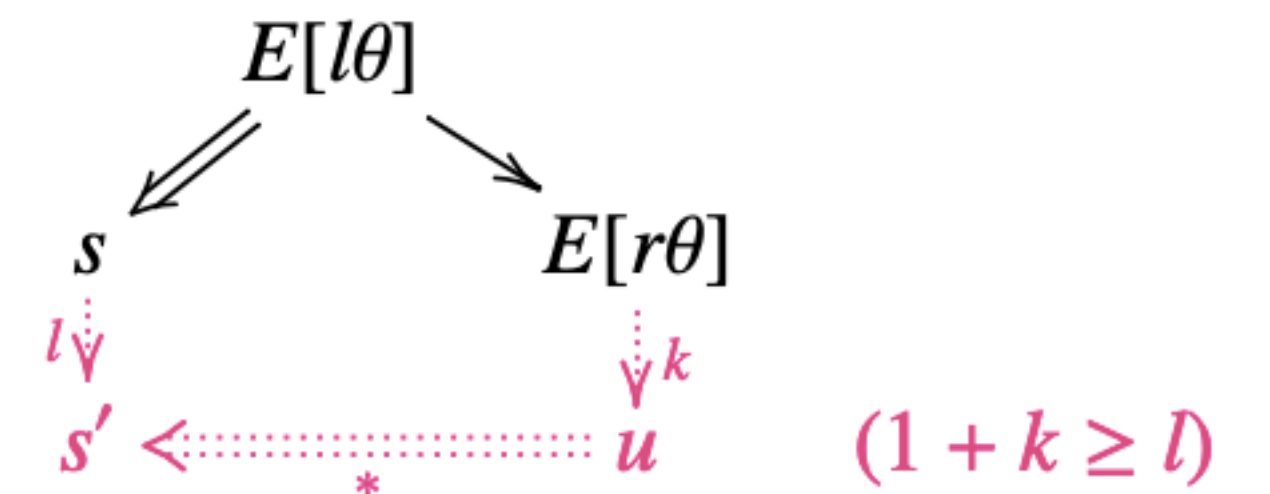
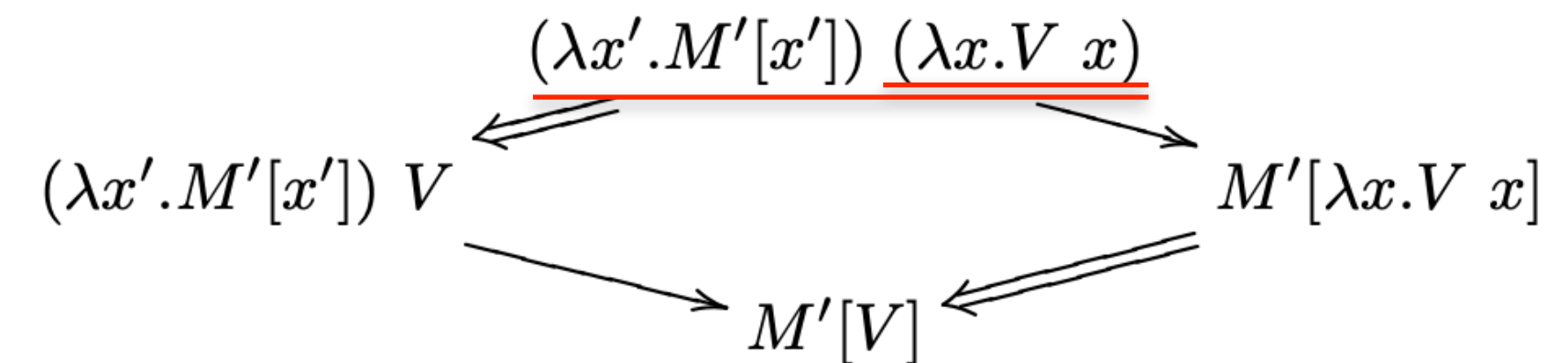
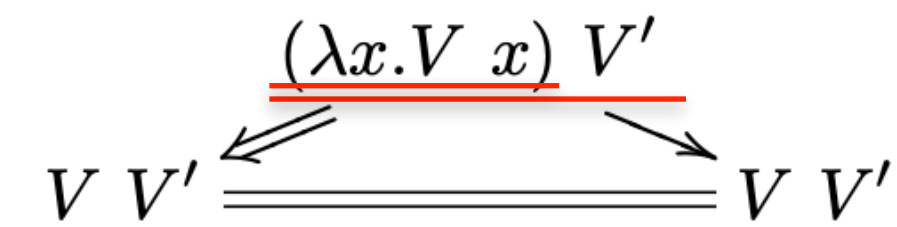
- e.g. call-by-value lambda-calculus as a core of functional programming

- beta-reduction as *computation step*

$$(\lambda x . M) \ V \rightarrow M[V/x]$$

- eta-reduction as *optimisation*

$$\lambda x. V \ x \Rightarrow V$$



- Goal: judge if *refinement* is “correct” wrt. *evaluation*

- Key sufficient condition: *local coherence* [Huet '80] [Aoto & Toyama '12]

can be checked by *critical pair analysis*

Graph evaluation and refinement systems

- systems with two kinds of rewriting
 - *evaluation* ($l \rightarrow r$) as specification of operational semantics
 - *refinement* ($l \Rightarrow r$) as optimisation
- Goal: judge if *refinement* is “correct” wrt. *evaluation*
 - namely, prove *contextual improvement* [Sands '96]

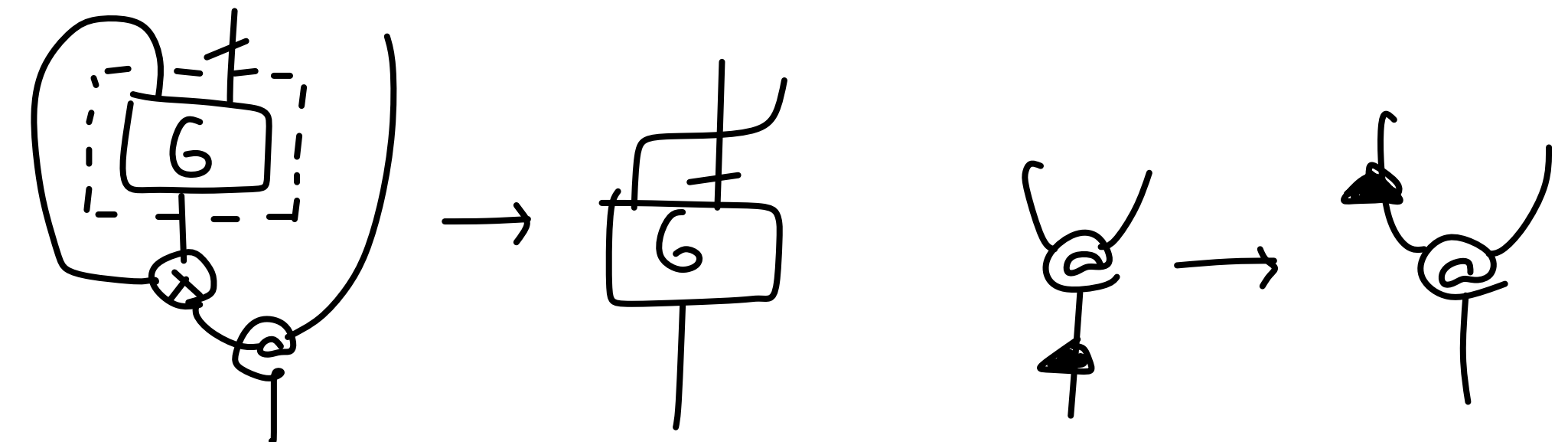
$$\forall C . \forall (l \Rightarrow r) . C[l\theta] \rightarrow^k v \implies C[r\theta] \rightarrow^m v \wedge k \geq m$$

Graph evaluation and refinement systems

- e.g. call-by-value lambda-calculus as a core of functional programming

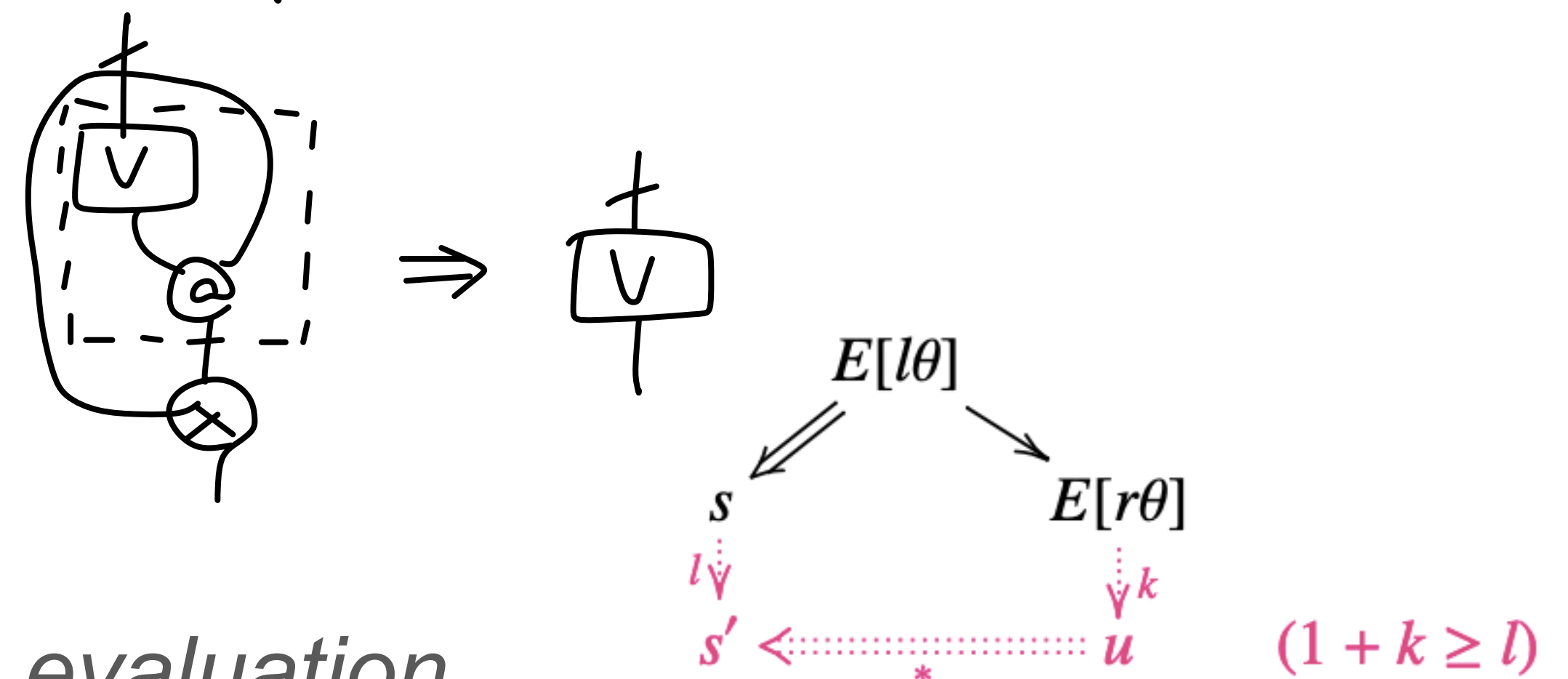
- beta-reduction as *computation step*

$$(\lambda x . M) V \rightarrow M[V/x]$$



- eta-reduction as *optimisation*

$$\lambda x . V x \Rightarrow V$$



- Goal: judge if *refinement* is “correct” wrt. *evaluation*

- Key sufficient condition: *local coherence* [Huet '80] [Aoto & Toyama '12]

can be checked by **critical pair analysis**